ResearchMind AI

Multi-agent research intelligence over a corpus of scientific papers

Razorpay AI Builder Internship 2026 · Open Track
Utkarsh · utkarshvats4108@gmail.com
github.com/utkarsh12377/ResearchMind-AI

A researcher asking “which of these approaches actually works best on this benchmark, and where do the papers disagree?” cannot answer it from any single paper. The answer lives in the relationships between them. ResearchMind AI ingests a corpus, builds a knowledge graph over it, and answers questions that require reading across all of it — citing every claim and saying so when it is not sure.

Form answers

PROJECT NAME / TITLE

ResearchMind AI — Multi-Agent Research Intelligence over Scientific Literature

PROJECT OBJECTIVES — WHAT DOES IT SOLVE?

Literature review does not scale. Answering one comparative question across 200 papers takes weeks, and existing “chat with your PDF” tools answer from a single document, cannot tell you when two papers contradict each other, and give you fluent prose with no way to check it.

ResearchMind AI targets four specific failures:

- **Retrieval that misses.** Semantic search alone loses exact identifiers like `WikiSQL`; keyword search alone loses paraphrase. Both run, get fused by reciprocal rank, and get reranked by a cross-encoder. A third retriever walks the knowledge graph, because “which other papers used this dataset” is a path, not a passage — no embedding finds it.
- **Answers you cannot check.** Every claim carries a citation marker resolved back to a specific passage. A verifier agent re-reads the answer against its sources and flags unsupported claims, and a flagged answer is capped at low confidence no matter how good retrieval looked.
- **Questions no single paper answers.** Comparison tables grouped by dataset and metric, contradiction detection, trend analysis over time, research gap discovery, and a cited literature review exportable as Markdown and BibTeX.
- **Demos that only work on the demo.** Every layer is an interface with an offline implementation, so the whole system runs and is tested with no containers, no API keys, and no external services.

GITHUB REPOSITORY URL

https://github.com/utkarsh12377/ResearchMind-AI

5-MIN PITCH VIDEO LINK

```
[ paste your video link here before submitting ]
```

Build challenges & technical obstacles

The problems below are the ones that actually cost time. Several were found by running the system end to end rather than by any unit test, which is the through-line: tests written against mocks would have passed for every one of them.

1. Half of hybrid search was silently dead

Uploading a paper indexed it into the vector store and nowhere else. The BM25 index was only rebuilt at startup, so every paper uploaded after boot was invisible to keyword retrieval until the next restart. Every test passed — they seeded the index directly and never exercised the upload path.

Caught by a live run where a freshly uploaded paper returned `sparse_rank: null` on every hit. The fix made the indexer write to both retrievers atomically. The lesson stuck: the ranks are now returned in the API response, so the same failure would be visible rather than silent.

2. Uploads returned 500 after succeeding

`asyncio.run() cannot be called from a running event loop`. Running Celery in eager mode for local development meant the ingestion task executed inside the API's own loop. The paper was stored, parsed, and indexed correctly — then the response blew up.

Fixed by detecting a running loop and dispatching to a single-worker thread pool when one exists. The subtlety is that the bug only appears in the configuration used for development, so it would have shipped fine and broken every contributor's first run.

3. Letting a model write database queries

Natural-language graph search means an LLM generating Cypher. The obvious mitigation — block dangerous keywords — loses to the first spelling you did not anticipate.

Built a whitelist validator instead: allowed clauses, allowed labels, allowed relationship types, mandatory LIMIT, single statement only. String literals are blanked before the keyword scan, so a paper legitimately titled "Deleting Noisy Labels" is not read as a DELETE — and equally, a keyword hidden inside quotes cannot execute. Its tests are written as attempts to get something past it rather than as happy-path coverage.

4. Retrieved text is an injection vector

A model reads retrieved passages as if they were instructions. There is no single fix, so the defence is layered: zero-width and bidirectional control characters stripped (invisible in a UI, meaningful to a tokenizer), external content wrapped in explicit UNTRUSTED delimiters, pattern-matching content dropped rather than forwarded, and tools that take typed arguments so no generated string reaches a shell or an arbitrary URL.

The judgement call: content tripping the detector is discarded, not passed through with a warning. Forwarding it would make the system's safety a property of how well the model resists persuasion.

5. A tokenizer bug that quietly deflated every quality metric

The evaluation metrics tokenize with a character class allowing dots, so `0.89` and `rouge-l` survive intact. It also meant a sentence-final `"encoder."` never matched the same word written mid-sentence — so faithfulness and relevancy were understated across the board.

Found because one assertion produced 0.5 where the arithmetic said 0.75. The tempting move was to lower the threshold; the right one was to ask why the number was wrong. Trailing punctuation is now stripped after matching.

6. The comparison table was empty on data that was obviously comparable

Comparison groups results on the (dataset, metric) pair. Rule-based extraction set model and metric but never a dataset, so offline every extracted row was ungroupable and the feature returned nothing on a corpus visibly full of comparable numbers.

Fixed by attributing a dataset from the known-entity lexicon — but only within roughly a sentence of the metric. Matching across the whole document was the tempting version, and is exactly how this kind of extractor ends up confidently attributing a number to whichever benchmark was named first. Model attribution deliberately stays with the LLM, because that genuinely does have to be read out of the prose.

7. Authorization that leaks through result counts

The natural way to scope retrieval is to rank everything and filter afterwards. That leaks: result counts, score distributions, and reranker behaviour all shift measurably in the presence of documents the caller cannot read.

The accessible set is resolved first and the candidate set restricted to it, so another workspace's content is never scored at all. The same rule governs the insight endpoints — passing someone else's paper id narrows the scope to nothing rather than returning a distinguishable error.

8. Observability that would have taken down the metrics store

Auto-instrumentation labels HTTP metrics by request path, which opens one Prometheus time series per paper id. Cardinality is the failure mode that kills a Prometheus install.

Instrumentation is hand-written: routes labelled by path template, statuses bucketed by class, nothing labelled by user input. Latency buckets are tuned for this workload too — retrieval lands in tens of milliseconds and a full agent run takes tens of seconds, so the library default topping out at 10s would have put every agent run in the overflow bucket. A test asserts that paper ids never appear in the metrics output.

9. Line endings that broke the container

Building on Windows produced `bad interpreter: /bin/sh^M` inside a Linux container. A CRLF shebang. The error names the shell rather than the line ending, which makes it a genuinely confusing hour. Fixed permanently in `.gitattributes` rather than by hand.

10. Building against a framework newer than my references

Next.js 16 moved several APIs, and shadcn on base-ui replaced the `asChild` composition pattern with a `render` prop. Rather than guessing from memory, I read the version's own bundled documentation. Its React Compiler lint rules then caught a real defect of mine — a synchronous `setState` inside an effect in the graph simulation — which I fixed rather than suppressed.

What got built

- **Ingestion** — layout-aware PDF parsing, OCR routed only to pages with a thin text layer, table and figure extraction with caption matching, bibliography parsing with DOI/arXiv resolution, and chunking that follows the paper's own section structure and carries breadcrumb paths.
- **Retrieval** — dense embeddings and a hand-written BM25 index fused by reciprocal rank, then cross-encoder reranking over an over-fetched candidate set. Every result reports which retriever found it and where reranking moved it.
- **Reasoning** — multi-provider LLM gateway with fallback and token accounting, token-budgeted context packing that compresses rather than truncates, and the Self-RAG / Corrective-RAG loop: grade retrieval, rewrite and re-retrieve if weak, answer, verify.
- **Agents** — ten agents in a LangGraph graph with a bounded critic↔reflector revision cycle, streaming each step to the UI as it completes.
- **Knowledge graph** — closed-schema entity extraction, GraphRAG expansion, natural-language Cypher behind the validator, and co-authorship, citation, and shared-entity network views.
- **Cross-paper analysis** — comparison tables, trend timelines, numeric and claim-level contradiction detection, gap discovery, and cited literature review generation with Markdown and BibTeX export.
- **Operations** — Prometheus metrics, Grafana dashboards and alert rules, an evaluation benchmark gating CI on retrieval quality, Kubernetes manifests with a migration Job, and a deploy pipeline that rolls back on failure.

Engineering decisions worth defending

- **Reciprocal rank fusion over weighted score blending.** Cosine similarity and BM25 scores are on incompatible scales, and normalising them requires knowing each distribution — which shifts with the corpus. RRF uses only the ordering, which is the part that is actually comparable.
- **Over-fetch before reranking.** Reranking exactly *k* results can only shuffle them. It cannot recover a relevant passage the first stage ranked at *k+1*, so the candidate set is four times the requested limit.
- **Validate model output before it becomes state.** Extraction is the one place a hallucination becomes durable, so unknown labels, unknown relation types, and edges between the wrong kinds of node are dropped rather than written.
- **Bound the agent revision loop.** A critic and a reviser left to argue will burn tokens indefinitely. The cycle is capped inside the critic.
- **Fallback is not a retry.** The LLM gateway falls back on provider outages but never on 4xx — a malformed request or a content-filter refusal fails identically everywhere. Streaming never falls back at all, because tokens already sent cannot be retracted.
- **Skip, do not fail, what cannot be measured.** Benchmark cases needing a real model are skipped offline rather than failed. A CI gate that is permanently red because of a missing key stops being read.

Verification

- **605 backend tests**, all passing, with no containers or API keys required. Tests build real PDFs with PyMuPDF and push them through the real ingestion, indexing, and retrieval pipeline — which is how four of the bugs above were found.
- **Retrieval benchmark gates CI** on faithfulness, context precision, and citation validity, ingesting a synthetic corpus through the real pipeline so a regression in chunking or fusion fails even when every unit test passes.
- **Live end-to-end run** across 26 checks: ingestion, upload rejection, request correlation, hybrid search with both retrievers contributing, all three graph views, comparison, trends, gaps, review generation, the full agent graph, and metric cardinality.
- **Clean** ruff, eslint, `tsc --noEmit`, and `next build`; all migrations round-trip up and down.

Stated honestly: Docker was not available on the build machine, so Postgres, Redis, Qdrant, Neo4j, real LLM providers, and Tesseract OCR are code-complete and contract-tested but were not smoke-tested against live instances. Everything verified above ran against SQLite, the offline embedding and LLM providers, and the in-memory vector and graph stores. The pluggable interfaces are exactly what makes that substitution possible — and exactly what makes the untested paths a real, named risk rather than a hidden one.